Homework 1

Operating Systems • Inha University in Tashkent • Fall 2026

Assigned	18.09.2026	**Instructor**	Yusupov Jalolliddin
Due	30.09.2026, 23:55 (UTC+5)	**Weight**	100 points / 40 = (2.5%)
Submit	One ZIP with source code	**Work**	Individual

What you submit

One ZIP file, containing exactly this:

```
SOC3010_HW1_<StudentID>_<Surname>.zip
├── report.pdf          ← the answers, the screenshots, the photos of your handwriting
└── src/
```

The report is one PDF. Write it in Word or LibreOffice (**using the template provided**) and export to PDF. Number your answers exactly as this handout numbers the questions — 1.1, 2.3, 5.6 — so we can find them.

The rules for screenshots

Most of the marks are for screenshots. A screenshot only counts if **all three** of these are true:

1. It is a picture of **your own terminal on your own VM**. The shell prompt must be visible — the part that looks like `student@ubuntu:~$`. Crop the window if you like, but never crop the prompt away.
2. The **command you typed is visible**, not only its output. We need to see what produced the result.
3. It is **readable**. A photograph of your monitor taken with a phone is not readable. Use your VM's screenshot key (PrtSc) or the Screenshot application.

A screenshot with no prompt in it is worth zero marks. It could have come from anyone.

The rules for handwriting

Some exercises are done on paper, by hand. Write your name and student ID at the top of every sheet, photograph the sheets, and put the photographs in the report. Typed Gantt charts and typed process graphs score zero. Drawing by hand is how this becomes fast for you in the exam, and the midterm will ask for exactly these two skills.

Part 1 — Try it on your machine **[10 points]**

Nothing to invent here. You are running programs you have already seen in the lectures, and reporting what your own machine does. Type the programs in yourself — do not paste them — because the typing is where you notice the semicolons.

> **Compiling.** All the C in this assignment is compiled the same way, on your Ubuntu VM: `gcc -Wall -o progname progname.c` then `./progname`
> `-Wall` means "warn me about all the suspicious things you notice". If `gcc` prints warnings, fix them before you go on — they are almost always a real bug.

1.1 — Signals arriving at a running program

Type in and run multi_signal.c from Lecture 2. It installs one handler for three different signals, prints its own PID, and then waits in pause().

With the program running, open a **second terminal** and send it signals. Record what happens in each case:

1. `Ctrl-C` in the first terminal
2. `kill -TERM <pid>` from the second terminal
3. `kill -USR1 <pid>`
4. kill -9 <pid>

Hand in: one screenshot showing both terminals (or two screenshots), and a four-line answer — for each signal, did the handler run, and did the program survive?

1.2 — fork() inside a loop

Run Program B from Lecture 3, but change the loop to run **four** times instead of three:

```
for (int i = 0; i < 4; i++) {
    fork();
}
printf("done\n");
```

Hand in: a screenshot, and two numbers — how many processes exist by the end, and how many times done is printed.

1.3 — exec replaces the program

Type in the `execlp` example from Lecture 3 — the one that prints its own PID, announces it is about to become `ls -l`, calls `execlp("ls", "ls", "-l", NULL)`, and has one more line of code after the `execlp` call.

Hand in: a screenshot. Then answer: **the line after `execlp` did not run. Why not?** And: was a new process created?

1.4 — Bash functions and return

In Lecture 4 you saw bash functions: you write hello() { ... } and then call it by its bare name. Here is the part we did not get to: a function can **return a status number**, exactly like a command does, and you read it with $? — the same $? from the exit-code table.

Notice that the function below is never given anything. It simply reads the variable num, because **a function and the script around it share the same variables** — whatever num holds when you call the function is what the function sees.

```
#!/bin/bash
# try_return.sh

is_big() {
    if [ $num -gt 100 ]; then
        return 0        # 0 means success — and "true" to if
    else
        return 1        # anything else means failure
    fi
}

num=500
is_big
echo "num is $num, is_big gave $?"

num=7
is_big
echo "num is $num, is_big gave $?"

# Because it returns a status, a function can be tested directly by if:
num=250
if is_big; then
    echo "$num is big"
else
    echo "$num is not big"
fi
```

Run it, then change the last num=250 to num=3 and run it again, so you see both branches.
Hand in: a screenshot of both runs, and one sentence answering: **why does return 0 mean success, when in most languages 0 means false?**

A function's `return` value is a status, not a result. It goes to `$?`, and `$?` is what `if` reads.

Part 2 — Process graphs, by hand **[20 points]**

This part is **handwritten on paper** and photographed. It is the single most exam-relevant thing in this assignment: the midterm will ask you to do exactly this, with no computer.
Use the method from Lecture 3. Nodes are the operations (`write, fork, wait, exit`). A horizontal line is one process's timeline. A vertical arrow is a `fork()`. A dashed edge is a `wait()`, forcing an order.

2.1 — Draw and count

```c
/* q21.c */
#include <stdio.h>
#include <unistd.h>
int main(void) {

    printf("A\n");
    fork();
    printf("B\n");
    fork();
    printf("C\n");
    fork();
    printf("D\n");
    return 0;
}
```

By hand: draw the process graph. Then state how many processes exist at the end, how many times **each** letter is printed, and the total number of lines. Then compile and run it, and put a screenshot next to your drawing. If your prediction and the machine disagree, the drawing is what we mark — but say which one was wrong and why.

2.2 — List every possible output

```c
/* q22.c */
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
int main(void) {
    printf("P\n");
    if (fork() == 0) {
        printf("C\n");
        exit(0);
    }
    printf("Q\n");
    wait(NULL);
    printf("R\n");
    return 0;
}
```

By hand: draw the graph, then **list every output this program can legally produce** — not the one you happened to see. For each ordering, give the one-line reason it is allowed.

2.3 — What wait() takes away

Now delete the `wait(NULL);` line from 2.2 and call it q23.c. Everything else is identical.
By hand: list every output **this** version can produce. Then answer: **which ordering became possible that was impossible before, and which edge of the graph disappeared?**
Finally, run q23 **twenty times** and record how many times you saw each ordering. Put the screenshot in if something changes.

Part 3 — The shell **[30 points]**

Everything in this part uses only what Lecture 4 covered. You do not need while, case, or any construct you have not seen. Every script starts with a shebang and is made executable with chmod +x.

3A — Two warm-up scripts

user.sh — asks for a login name with read, then compares it with the environment variable $USER. If they match, print Hello, <name>. How are you today?. If they do not, print You are not <user>, so who are you?.

number.sh — asks for a number and checks that it is strictly between 1 and 10. If it is, print <n>*<n>=<n squared>, computing the square with shell arithmetic. If it is not, print Wrong insertion !.

Write number.sh with **one single if** using a logical operator, not two nested ifs.

Hand in: both scripts in src/, and one screenshot per script showing it run twice — once with input that passes, once with input that fails.

3B — jobctl.sh

This is the substantial one. Write a single script, jobctl.sh, that a person could actually use to launch a few background jobs and look at them. It is run like this:

```
./jobctl.sh /tmp/hw1logs alpha beta gamma
```

The first argument is a log directory. Every argument after it is the name of a job to start. Your script must do all eight of the following. Each is worth marks on its own, so implement as many as you can even if one defeats you.

#	Requirement	Points
1	**File tests.** If the log directory does not exist, create it with mkdir. Then check that it really is a directory and that it is writable. If either test fails, print why and exit 1. Use at least three different file test operators.	2
2	**A function that returns a status.** Write valid_name(), which reads the variable name — exactly like is_big in 1.4, with nothing passed to it — and returns 0 if that name is usable and 1 if it is not. A name is unusable if it is **empty**, or if a job with that name **already exists**, that is, the directory <logdir>/<name> is already there.	2

3	**Loop over the job names.** Loop over the arguments, **skipping the first one** (it is the log directory, not a job). Each time round, put the current argument into the variable `$name`, then call valid_name and test it with if. For each accepted name create its directory with mkdir; for each rejected one print a warning saying why.	2
4	**An array.** Keep an array of the names you accepted — it starts empty and grows as the loop runs — and print the whole array at the end with ${accepted[*]}. See the note below on starting an array empty.	2
5	**Arithmetic.** Count accepted and rejected names in two variables using shell arithmetic, and print a summary line: accepted 3, rejected 1, total 4 — with the total computed, not typed.	2
6	**Background jobs.** For every accepted name, start a sleep 120 in the background with &. Then run jobs so the script's own job table is printed.	2
7	**Exit codes.** Run three things in turn and print $? after each, with a label, so that you produce **0**, **127** and **126**. For 126, remember that Lecture 4 said 126 means "found, but not executable — no x bit, *or a directory*": so mkdir a directory inside your log directory and then try to run it as a command.	2
8	**trap.** Install a trap on SIGINT that prints a message, and a trap on EXIT that prints jobctl.sh finished. The EXIT trap must fire on every path out of the script, including the exit 1 ones.	1

Hand in: jobctl.sh in src/, plus screenshots showing (a) a successful run where at least one name is accepted and one is rejected, (b) a run with no arguments at all, (c) the exit-code section clearly showing 0, 127 and 126.

Starting an array empty. Lecture 4 showed two ways to make an array: one element at a time, `pet[0]=dog`, or all at once, `pet=(dog cat fish)`. Here you need a third, because you do not know the names in advance — you are collecting them as the loop runs:

```
accepted=()            # empty brackets: the array exists, but has no elements
yet
accepted[0]="alpha"    # now it has one
accepted[1]="beta"     # now two
echo "${accepted[*]}"  #  →  alpha beta
```

In the loop you will not write the index by hand — you will use your counter variable, `accepted[$good]="$arg"`, which is why the counter in requirement 5 does two jobs at once.

3C — A job-control session, by hand

This is done in a terminal session that you drive yourself and screenshot. Do it in one terminal, in this order:

1. Start sleep 60 in the foreground and **kill it with Ctrl-C**. Then run echo $?. Write down the number you get, and show in your report how it is built out of 128 and a signal number.
2. Start sleep 300 in the foreground. Suspend it with Ctrl-Z.
3. Run jobs. Note the job number.
4. Send it to the background with bg.
5. Start two more: sleep 400 & and sleep 500 &.
6. Run jobs again. You should see three.
7. Bring the second one to the foreground with fg %2, then suspend it again with Ctrl-Z.
8. Run `ps -o pid,ppid,pgid,sid,stat,comm`. Record the PGID of each of the three sleep processes. Are they the same or different?.
9. Choose one of the sleep processes and kill its whole process group with `kill -INT -<PGID>`.
10. Run `jobs` one last time. Which job disappeared?
11. Last of all, redo the export experiment from Lecture 4 **by hand**, typing each line and watching what the child shell can see:

```
x=hello
bash            # you are now inside a child shell
echo $x         # ?
exit            # back to the parent shell
export x
bash
echo $x         # ?
x=ciao
exit            # back to the parent again
echo $x         # ?
```

Hand in: screenshots of steps **1, 6, 8, 10 and 11**, and answers to these four questions:

1. In the ps output at step 7, what is in the STAT column for a suspended job, and what is there for a running background job? What do those letters mean?
2. Do all three sleep processes share one PGID?
3. What did the minus sign in kill -INT -<PGID> change? What would have happened without it?
4. In step 11, the last echo $x printed hello, not ciao — even though you had just set x=ciao. Why did the child's change not come back to the parent?

Part 4 — Zombies, and how to stop making them **[20 points]**

Two C programs. The first one makes zombies on purpose. The second one is the same program with the fix, and you have to prove the fix works.

> **Seeing a process's children.** `ps -o pid,ppid,stat,comm --ppid <PID>` lists only the children of the process you name, which is much easier to read than the whole system. A child in state **Z**, shown as `<defunct>`, is a zombie.

4.1 — zombie.c: make the problem

Write a program that:

- prints its own PID with getpid(), so you can watch it from another terminal;
- forks **four** children; each child sleeps a different short time (1, 2, 3, 4 seconds), then calls exit(0);
- the parent does **not** call wait or waitpid at all — it just calls sleep(30) and then exits.

Run it. While it is still in that 30-second sleep, go to a second terminal and run
`ps -o pid,ppid,stat,comm --ppid <the PID it printed>`

Hand in: the screenshot showing four processes in state **Z**

4.2 — reaper.c: fix it with a signal handler

Copy zombie.c to reaper.c and add a SIGCHLD handler, installed with **sigaction**, which reaps the children as they die. Requirements:

Requirement	Points
The handler is installed with sigaction, not signal. You must set sa_handler, clear the mask with sigemptyset(&sa.sa_mask), set sa_flags = SA_RESTART, and check that sigaction() did not return -1.	4
The handler reaps with the loop form from Lecture 3: while (waitpid(-1, NULL, WNOHANG) > 0) { … } — and counts how many children it reaped.	3
A second handler on SIGINT, also installed with sigaction, which sets a flag variable instead of doing the work in the handler. The main program must check the flag and exit cleanly when it is set.	2
The program reports how many children it reaped before it exits.	1

Hand in: reaper.c, plus the same `ps --ppid` screenshot as in 4.1, now showing **no** Z and no <defunct>.

4.3 — Three questions

1. Lecture 3 insisted on a while loop in the SIGCHLD handler rather than a single waitpid. Explain, in two sentences, what goes wrong with a single waitpid.
2. Give **two** of the four reasons Lecture 3 gave for preferring sigaction() over signal().
3. In zombie.c, the parent eventually exits after 30 seconds. What happens to the four zombies at that moment, and which process is responsible?

Part 5 — CPU scheduling, by hand **[20 points]**

On paper, by hand, photographed. No simulator, no program, no spreadsheet. For every question draw the Gantt chart first, then read the numbers off it.

For every process, unless the question says otherwise, **arrival time is 0**. Use these two definitions from Lecture 5, and show the division for each average:

Turnaround time = completion time − arrival time.
Waiting time = turnaround time − burst time. (Equivalently: the total time the process sat in the ready queue.)

5.1 — SRTF, preemptive

Process	Arrival	Burst
P1	0	8
P2	2	4
P3	4	1
P4	5	3

Hand in: Gantt chart, average waiting time, average turnaround time. Show the remaining-time table you used at each decision point, as in the lecture.

5.2 — Priority, preemptive (smaller number = higher priority)

Process	Arrival	Burst	Priority
P1	0	7	3
P2	1	4	2
P3	3	2	1
P4	6	3	4

Hand in: Gantt chart, average waiting time, average turnaround time. Then: which process here is at risk of **starvation** if more high-priority work keeps arriving, and what is the standard fix called?

5.3 — Round robin

All arrive at 0, served in the order P1, P2, P3, P4. **Quantum = 3 ms.**

P1 = 9, P2 = 4, P3 = 6, P4 = 2.

Gantt chart, average waiting time, average turnaround time.

Then: a process needs 10 ms of CPU. How many context switches does it take with a quantum of 12, with 5, and with 1? What does that tell you about choosing a quantum? NOTE: Count only preemptions caused by expiration of the time quantum; do not count the initial dispatch or final completion.

Submission checklist

1. The ZIP is named `SOC3010_HW1_<StudentID>_<Surname>.zip`.
2. `report.pdf` is inside it, and opens.
3. Every screenshot shows your shell prompt and the command you typed.
4. Parts 2 and 5 are **photographs of handwriting**, with your name and ID on every sheet.
5. `src/` contains every `.c` and `.sh` file, and each one compiles or runs.